

TENSOR DECOMPOSER

An Interactive High-Dimensional Multiway Data Analysis and Benchmarking
System

MIDTERM TECHNICAL REPORT SE 801 — Software Project Lab III

Submitted By:

Md. Kamrul Hasan
Roll: 1418 | Phone: 01535757066
BSSE 8th Semester
Institute of Information Technology
University of Dhaka

Supervised By:

Mohd. Zulfiquar Hafiz
Assistant Professor
Institute of Information Technology
University of Dhaka

Submitted To:

SPL 3 Program Committee
Institute of Information Technology
University of Dhaka
Date of Submission: 2 August 2026

Letter of Transmittal

Date: August 2026

To,

The SPL 3 Program Committee & Exam Committee
Institute of Information Technology (IIT)
University of Dhaka, Dhaka-1000

Subject: Submission of Midterm Technical Report on 'Tensor Decomposer'

Dear Sir/Madam,

I am pleased to submit the Midterm Technical Report for my Software Project Lab III (SE 801) project, titled "Tensor Decomposer: An Interactive High-Dimensional Multiway Data Analysis and Benchmarking System".

This report details the requirements analysis, system architecture, scenario-based system modeling, activity workflows, preliminary test plans, and implementation roadmap for the web-based tensor decomposition platform. The system incorporates higher-order multilinear algebraic algorithms—including CP Decomposition, Tucker Decomposition, HOSVD, and Tensor Train Decomposition, along with standard matrix factorizations—paired with single-run FLOPs benchmarking, error metrics analysis, and dynamic 3D visual representations.

I express my sincere gratitude for your guidance and continuous support throughout the project. I look forward to your valuable feedback and recommendations.

Sincerely,

Md. Kamrul Hasan

Roll: 1418

BSSE 8th Semester, IIT, University of Dhaka

Acknowledgement

First and foremost, I express my deepest gratitude to Almighty for providing me with the strength, endurance, and capability to complete the midterm milestones of this project successfully.

I extend my sincere appreciation to my project supervisor, Mohd. Zulfiquar Hafiz, Assistant Professor, Institute of Information Technology (IIT), University of Dhaka, for his invaluable guidance, scholarly advice, and constructive reviews throughout the formulation and implementation of this research project.

I am also profoundly grateful to the faculty members and SPL 3 Program Committee of the Institute of Information Technology for providing a rich academic environment and technical infrastructure that made this work possible.

Abstract

Modern datasets across scientific disciplines—such as deep learning, image processing, signal processing, neuroimaging, and chemometrics—are inherently multi-dimensional and naturally structured as higher-order tensors. Traditional matrix factorization methods like SVD or PCA require flattening (matricizing) multiway data, which severely destroys higher-order spatial and multiway structural correlations. Conversely, computing higher-order tensor factorizations introduces substantial computational complexity, memory overhead, and parameter explosion.

To bridge this gap, this report presents Tensor Decomposer, an interactive, web-based analytical platform designed to decompose, benchmark, and visualize higher-dimensional tensor datasets. The platform implements core tensor decomposition algorithms—specifically Canonical Polyadic (CP) Decomposition via Alternating Least Squares (CP-ALS), Tucker Decomposition via Higher-Order Orthogonal Iteration (HOOI/Tucker-ALS), Higher-Order Singular Value Decomposition (HOSVD), and Tensor Train (TT) Decomposition—alongside standard matrix factorizations (SVD, QR, LU, Eigendecomposition). The system incorporates high-precision single-run performance analytics (FLOPs estimation, computational complexity formulas, compression ratios) and reconstruction accuracy evaluations (MAE, RMSE, Relative Error). Furthermore, an interactive SVG/isometric visual engine renders 3D decomposition structures, component spectra, and matrix heatmaps. This report outlines the system requirements, scenario-based modeling, activity workflows, preliminary test plans, and implementation progress.

Table of Contents

Letter of Transmittal	2
Acknowledgement	3
Abstract	3
1. Project Overview	5
1.1 Project Title	5
1.2 Problem Statement	5
1.3 Objectives	6
1.4 Scope	6
1.5 Deliverables	7
2. Requirements Analysis	8
2.1 Functional Requirements	8
2.2 Non-Functional Requirements	10
2.3 Stakeholders	11
3. System Modeling	12
3.1 Use Case Diagram & Descriptions	12
3.2 Detailed Use Case Specifications	14
3.3 Activity Diagram & Description	18
4. Preliminary Test Plan	21
4.1 Testing Objectives	21
4.2 Features to be Tested	21
5. Timeline	25
4.1 Project Gantt Chart	25
4.2 Work Breakdown Schedule	26
6. Conclusion & Future Work	27
7. References	28

1. Introduction

1.1 Project Title

Tensor Decomposer: An Interactive High-Dimensional Multiway Data Analysis and Benchmarking System

1.2 Problem Statement

In modern data-driven fields such as computer vision, deep learning, signal processing, neuroimaging (EEG/fMRI), and chemometrics, data naturally manifests in multi-dimensional formats (tensors) rather than traditional 2D matrices. For example, color video sequences possess spatial, temporal, and chromatic dimensions (Height x Width x Frames x Channels).

When conventional matrix reduction techniques like Principal Component Analysis (PCA) or Singular Value Decomposition (SVD) are applied to higher-order data, the tensor must first be flattened into a 2D matrix. This matricization severely destroys intrinsic multiway correlations, spatial patterns, and multi-linear structure. Conversely, operating directly on raw high-dimensional tensors suffers from the 'curse of dimensionality', leading to exponential growth in computational memory, execution time, and storage overhead.

While tensor decomposition algorithms—such as Canonical Polyadic (CP) decomposition, Tucker decomposition, Higher-Order SVD (HOSVD), and Tensor Train (TT) decomposition—offer powerful mathematical frameworks for multiway data compression and feature extraction, existing software solutions suffer from several critical shortcomings:

1. **Fragmented Implementations:** Algorithms are scattered across disparate language ecosystems (MATLAB Tensor Toolbox, Python TensorLy, C++ packages) without a unified interface.
2. **Absence of Real-Time Analytics:** Existing tools lack automated single-run computational benchmarking (FLOPs estimation, complexity formulas, compression ratios).
3. **Lack of Interactive Visualizations:** Researchers lack web-accessible visual representations to inspect decomposed core tensors, component factor spectra, and 3D equation layouts dynamically.

1.3 Objectives

The primary goal of the Tensor Decomposer project is to architect and deliver a unified web-based platform for interactive tensor decomposition, reconstruction accuracy analysis, performance benchmarking, and visualization. The specific technical objectives include:

- **Comprehensive Algorithm Suite:** Implement robust mathematical algorithms for tensor decompositions (CP-ALS, HOOI/Tucker-ALS, HOSVD, Tensor Train) and core matrix factorizations (SVD, QR, LU, Eigendecomposition).
- **In-Browser Interactive Interface:** Develop a responsive frontend capable of ingesting raw manual numerical tensors or uploaded data files (JSON/CSV) and rendering immediate analytical output.
- **Precision Reconstruction & Error Metrics:** Provide exact numerical evaluation of reconstruction quality

by calculating Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Relative Error, and reconstructed element head previews.

- Performance Benchmarking Engine: Compute single-run execution times (ms), floating-point operations (FLOPs count), theoretical complexity formulas, parameter compression counts, and compression ratios.
- Multi-Method Comparative Benchmarking: Enable side-by-side algorithmic comparisons across compression efficiency, reconstruction error, and computational runtime.
- 3D Isometric & Spectral Visualizations: Render scaled isometric 3D block diagrams, eigenvalue/singular value spectra, and matrix heatmaps dynamically using SVG.

1.4 Scope

The project encompasses the design, algorithm development, frontend visual engine, and backend web integration for the Tensor Decomposer application.

In Scope:

- Web application built with Django (backend service orchestration) and modern ES Modules JavaScript (frontend interactivity).
- Implementation of CP-ALS, Tucker-ALS (HOOI), HOSVD, Tensor Train, SVD, QR, LU, and Eigendecomposition algorithms.
- Interactive UI components featuring theme toggling (Dark/Light mode), mutual-exclusion inputs (manual text vs. file upload), and AJAX loading states.
- Calculation of compression ratios, parameter counts, FLOPs estimates, MAE, RMSE, relative errors, and preview heads.
- Dynamic generation of isometric SVG equations, bar chart spectra, and scalable matrix heatmaps.

Out of Scope (for current midterm release):

- Further Optimization for better output.

1.5 Deliverables

1. Complete Web Application Codebase: Django backend and modular ES Module frontend static JavaScript.
2. Core Mathematical Algorithm Suite: Fully tested Python modules for CP, Tucker, HOSVD, TT, SVD, QR, LU, and Eigendecomposition.
3. Interactive Visualization Engine: Client-side SVG graphics generator for 3D isometric equation layouts, spectra, and heatmaps.
4. Midterm Technical Report & System Documentation: Comprehensive software architecture, requirements analysis, system modeling, test plans, and project timeline.

2. Project Requirements (QFD)

2.1 Functional Requirements

The functional requirements of the Tensor Decomposer platform specify the core capabilities and services provided by the application.

Function ID	Feature Name	Description & Functional Specification
FR-01	Manual Tensor Input	The system shall allow users to manually enter tensor data in array format (1D, 2D, 3D, 4D).
FR-02	File Upload Ingestion	System shall accept file uploads containing tensor structures formatted in JSON or CSV.
FR-03	Mutual Exclusion Input	The system shall automatically clear file selection upon manual text edit, and vice-versa.
FR-04	Algorithm Selection	The system shall allow selection among CP, Tucker, HOSVD, TT, SVD, QR, LU, and Eigendecomposition.
FR-05	CP Decomposition (CP-ALS)	The system shall compute rank-R CANDECOMP/PARAFAC factor matrices and weight vectors using CP-ALS.
FR-06	Tucker Decomposition (HOOI)	The system shall compute core tensor and orthogonal factor matrices using Higher-Order Orthogonal Iteration.
FR-07	Tensor Train Decomposition	The system shall compute a sequence of 3D core tensors connecting tensor modes.
FR-08	Matrix Factorization Suite	The system shall perform SVD, QR, LU, and

		Eigendecomposition for 2D matrix inputs.
FR-09	Reconstruction Analysis	The system shall reconstruct the original tensor from decomposed factors and measure absolute & relative errors.
FR-10	Accuracy Metrics	The system shall calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).
FR-11	FLOPs Benchmarking	The system shall estimate total Floating Point Operations (FLOPs) and format in KFLOPs/MFLOPs/GFLOPs.
FR-12	Single-Run Performance	The system shall measure execution time (ms), compressed parameter counts, and compression ratios.
FR-13	Algorithmic Comparison	The system shall run multi-method benchmarking comparisons across all algorithms side-by-side.
FR-14	3D Isometric Visualization	The system shall generate 3D isometric equation blocks scaling based on tensor shapes.
FR-15	Theme Management	System shall support theme toggling (Dark/Light mode) persisting across browser sessions.

2.2 Non-Functional Requirements

NFR ID	Category	Quality Requirement Specification
NFR-01	Performance	Algorithm execution for small to medium tensors (< 100x100x100) shall return results within < 500 ms.
NFR-02	Reliability & Accuracy	Matrix and tensor decomposition algorithms shall maintain floating point accuracy within 1e-6 error bounds.
NFR-03	Usability & Responsive UI	The user interface shall be intuitive, mobile-responsive, and maintain high-contrast dark/light themes.
NFR-04	Maintainability & Quality	Frontend code shall follow modular ES Module standards, separating DOM logic, formatting, and SVG rendering.
NFR-05	Portability & Compatibility	The system shall function across modern web browsers (Chrome, Brave, Firefox, Edge, Safari) without external plugins.
NFR-06	Error Handling & Feedback	Invalid user inputs (e.g., non-numeric arrays, mismatched ranks) shall display informative error panels.

2.3 Stakeholders

1. Academic Researchers & Data Scientists: Primary users who utilize the system to compress higher-dimensional tensors, benchmark decomposition runtimes, and inspect component factor matrices.
2. Students & Educators: Users seeking interactive visual tools to understand multi-linear algebra concepts (CP vs. Tucker vs. HOSVD).
3. System Developers & Maintainers: Engineers responsible for expanding the algorithm suite, maintaining backend Python services, and optimizing web frontend components.
4. Project Supervisors & Academic Reviewers: Evaluators monitoring system adherence to software engineering standards, theoretical correctness, and performance benchmarks.

3. System Modeling

3.1 Use Case Diagram & Descriptions

The Use Case Diagram is modeled across three levels of detail, following a leveled decomposition approach: a system-level context view (Level 0), a module-level breakdown (Level 1), and a step-level breakdown of each module's internal workflow (Level 2). This mirrors how the functional scope of the Tensor Decomposer narrows from a single black-box interaction down to the concrete actions the backend performs.

3.1.1 Level 0: System Context

At the highest level, the entire system is represented as a single use case that the User interacts with directly, with no internal functionality exposed.

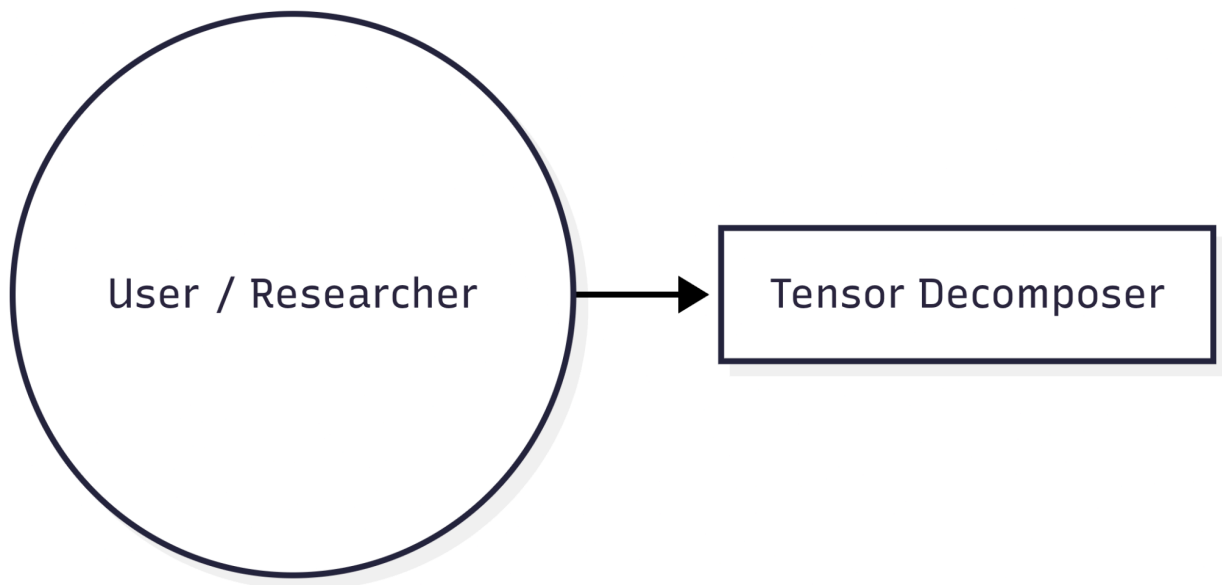


Figure 3.1a: Level 0 – System Context Diagram

Primary Actor: User / Researcher

Secondary Actors: None at this level

Goal in Context: Represents the high-level overview of the Tensor Decomposer as a single system boundary.

3.1.2 Level 1: Modules of Tensor Decomposer

The system-level use case is decomposed into five core modules, each representing a major functional capability of the platform. The User interacts with each module directly.

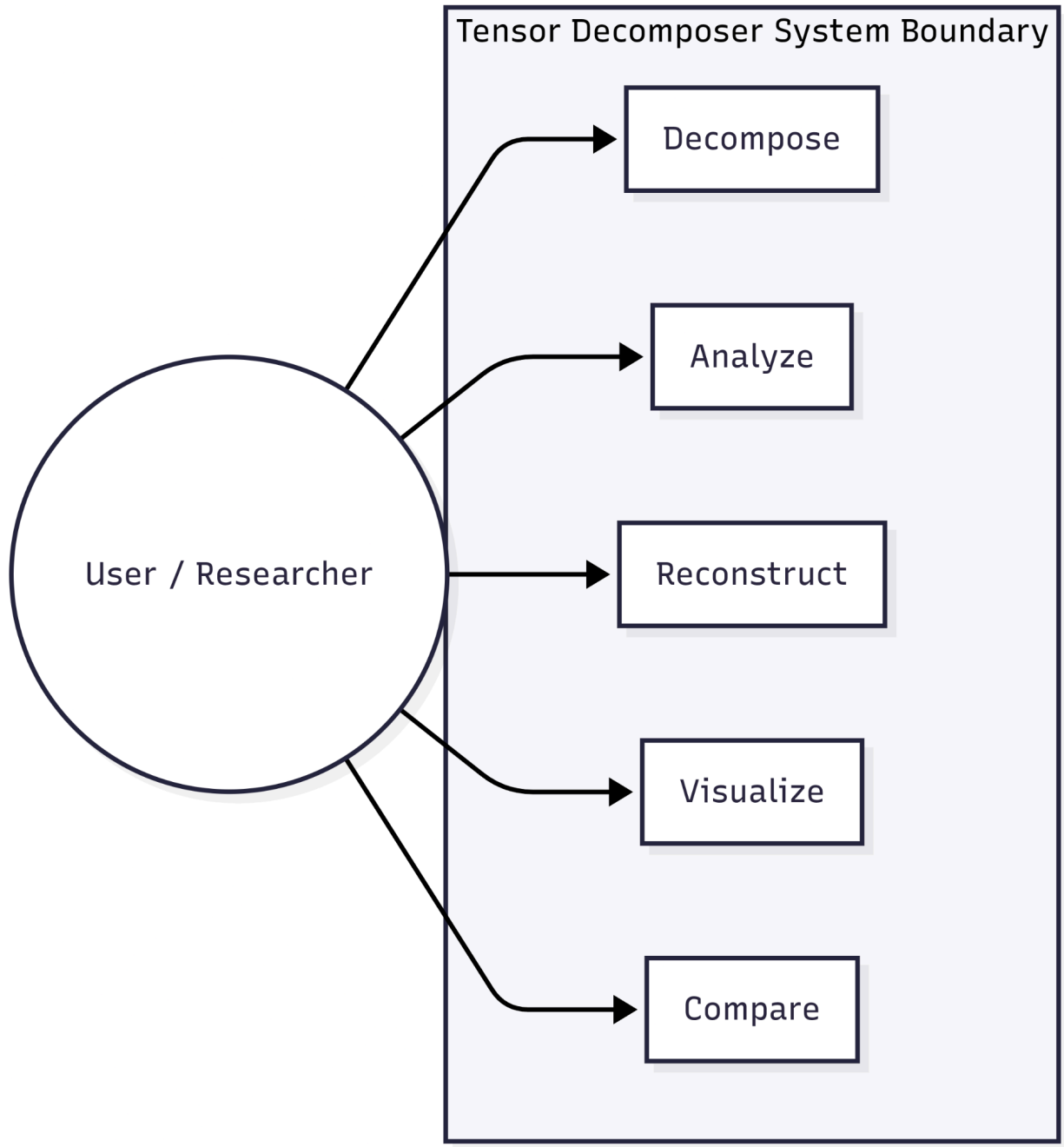


Figure 3.1b: Level 1 – Modules of Tensor Decomposer

Primary Actor: User / Researcher

Secondary Actors: None at this level

Goal in Context: Represents the five core modules — Decompose, Analyze, Reconstruct, Visualize, and Compare — that make up the Tensor Decomposer system.

3.1.3 Level 2: Module Breakdown

Each Level 1 module is further decomposed into its internal sequence of steps, showing the concrete actions performed as the User moves from providing input to receiving a result. The User connects to the first and last step of each sequence, consistent with how the module is entered and exited.

Decompose

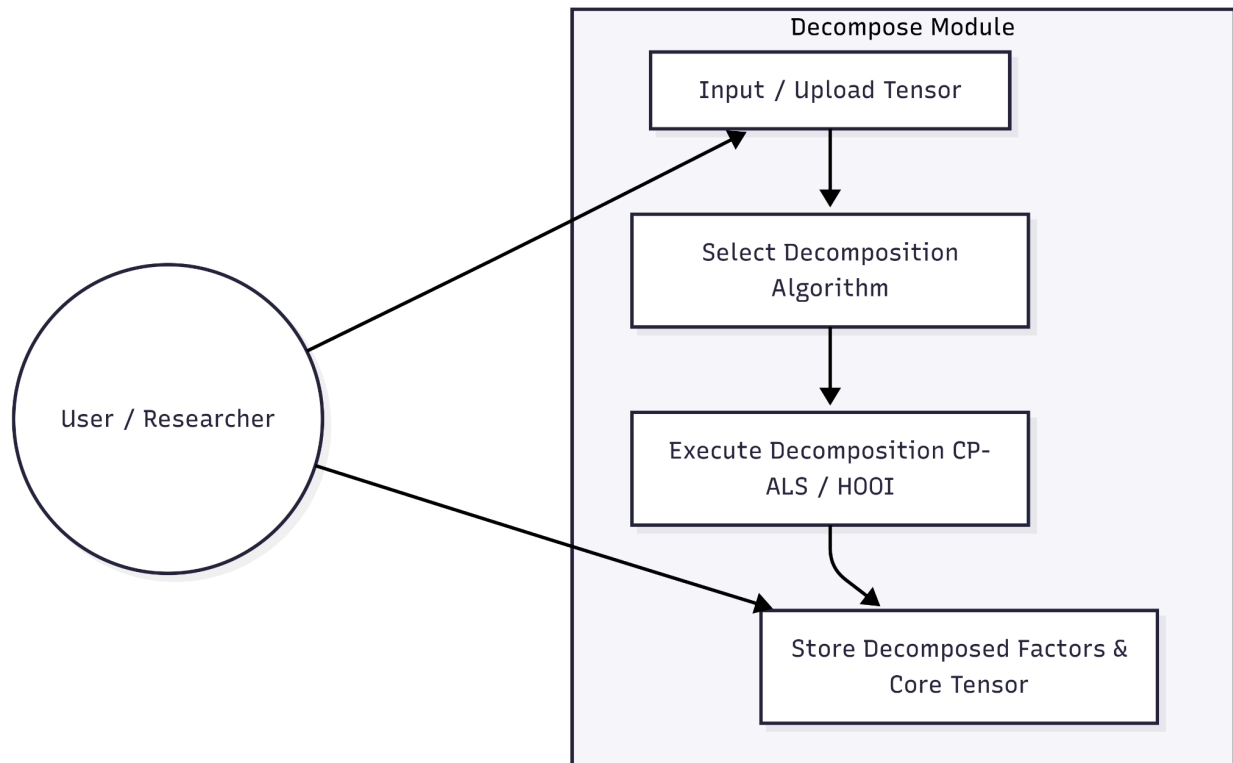


Figure 3.1c-1: Level 2 – Decompose Module

Primary Actor: User / Researcher

Goal in Context: Breakdown of the Decompose module — preparing input and running the selected decomposition algorithm.

1. User uploads or manually enters tensor data → system validates array dimensions and formatting.
2. The user selects a decomposition algorithm (CP, Tucker, HOSVD, TT, SVD, QR, LU) → system loads the corresponding routine.
3. User triggers execution → backend runs the algorithm (CP-ALS / HOOI).
4. The system stores the resulting factor matrices and core tensor for downstream use.

Analyze

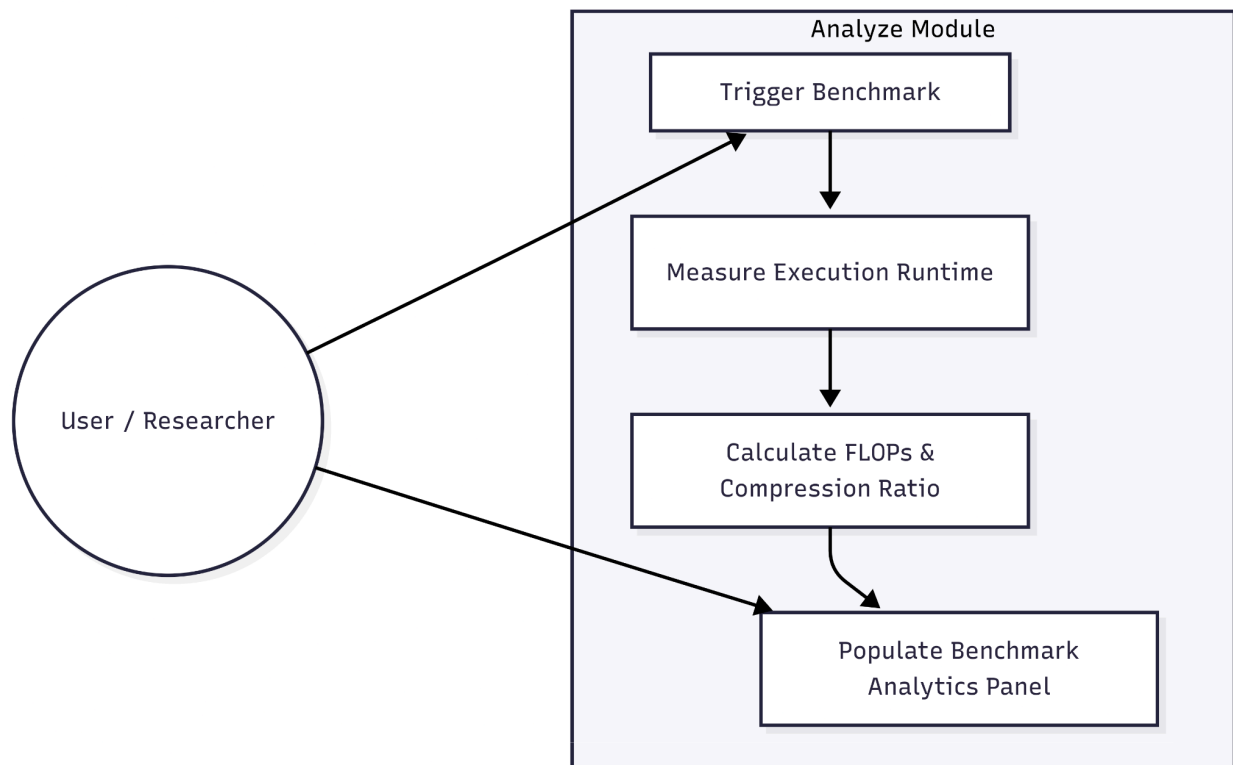


Figure 3.1c-2: Level 2 – Analyze Module

Primary Actor: User / Researcher

Goal in Context: Breakdown of the Analyze module — benchmarking the computational cost of a completed decomposition run.

1. The user triggers the Benchmark action → the system prepares to measure the current run.
2. The system measures execution runtime across iterations.
3. Backend calculates FLOPs count and parameter/compression ratio.
4. The system populates the benchmark analytics panel with the resulting metrics.

Reconstruct

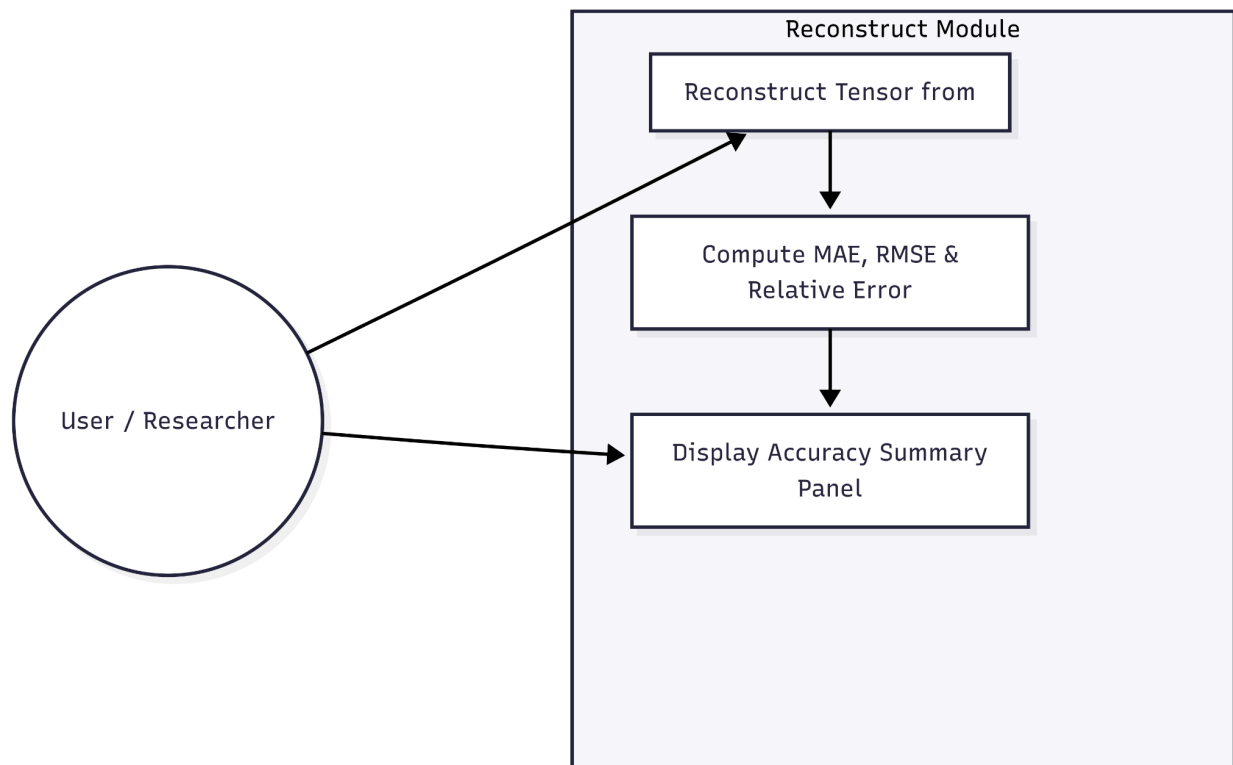


Figure 3.1c-3: Level 2 – Reconstruct Module

Primary Actor: User / Researcher

Goal in Context: Breakdown of the Reconstruct module — rebuilding the tensor and checking reconstruction accuracy.

1. The user clicks the Analyze/Reconstruct action → backend reconstructs the tensor from its factor matrices.
2. The system computes MAE, RMSE, and Relative Error against the original tensor.
3. The system displays the accuracy summary panel with a preview of reconstructed elements.

Visualize

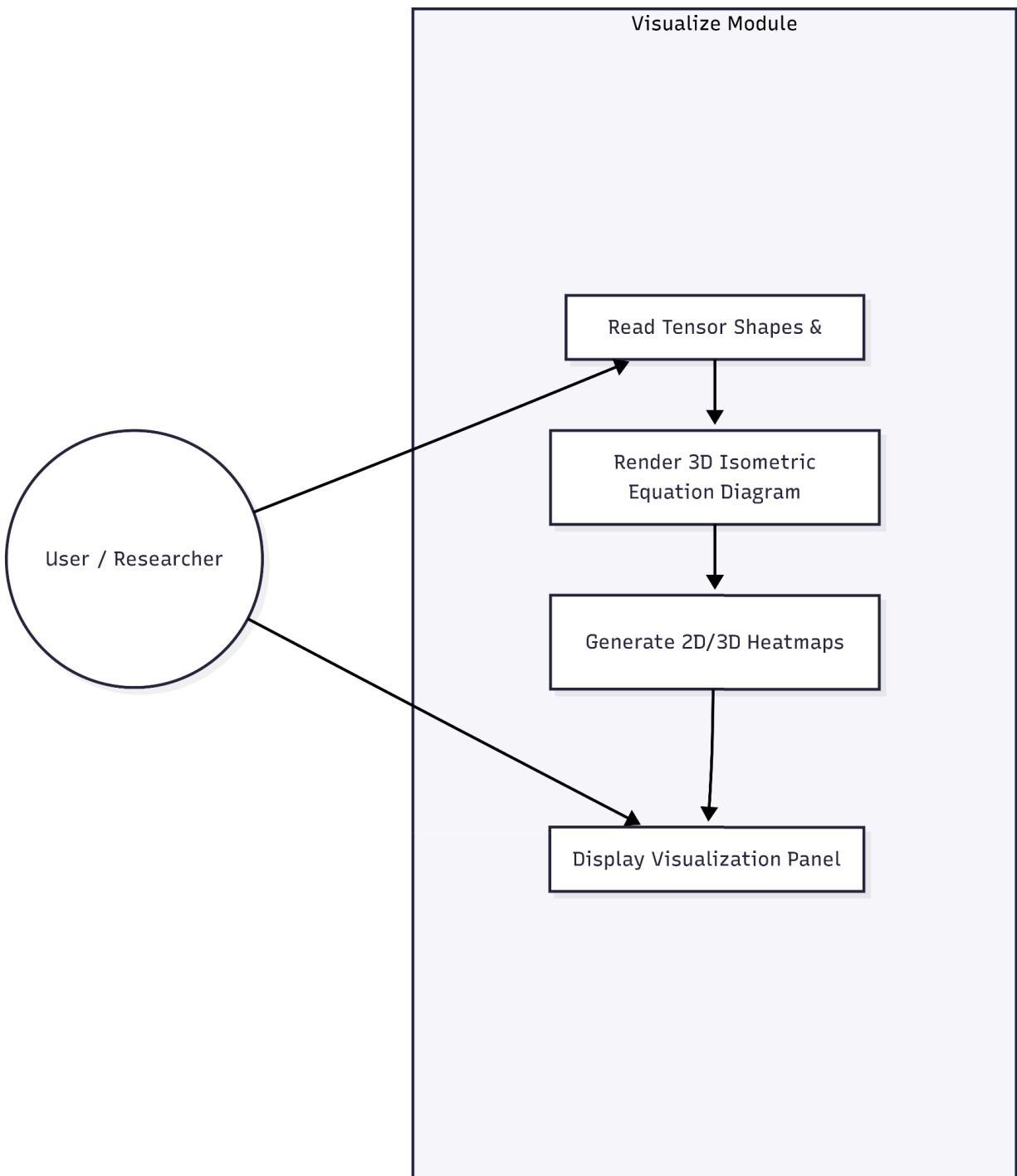


Figure 3.1c-4: Level 2 – Visualize Module

Primary Actor: User / Researcher

Goal in Context: Breakdown of the Visualize module — rendering decomposed and reconstructed outputs.

1. The user clicks the Visualize action → the front end reads tensor shapes and factor matrices.
2. SVG engine draws the 3D scaled isometric blocks for the equation structure.
3. Heatmap generator draws normalized 2D/3D slice matrix heatmaps.
4. The system displays the completed visualization panel to the user.

Compare

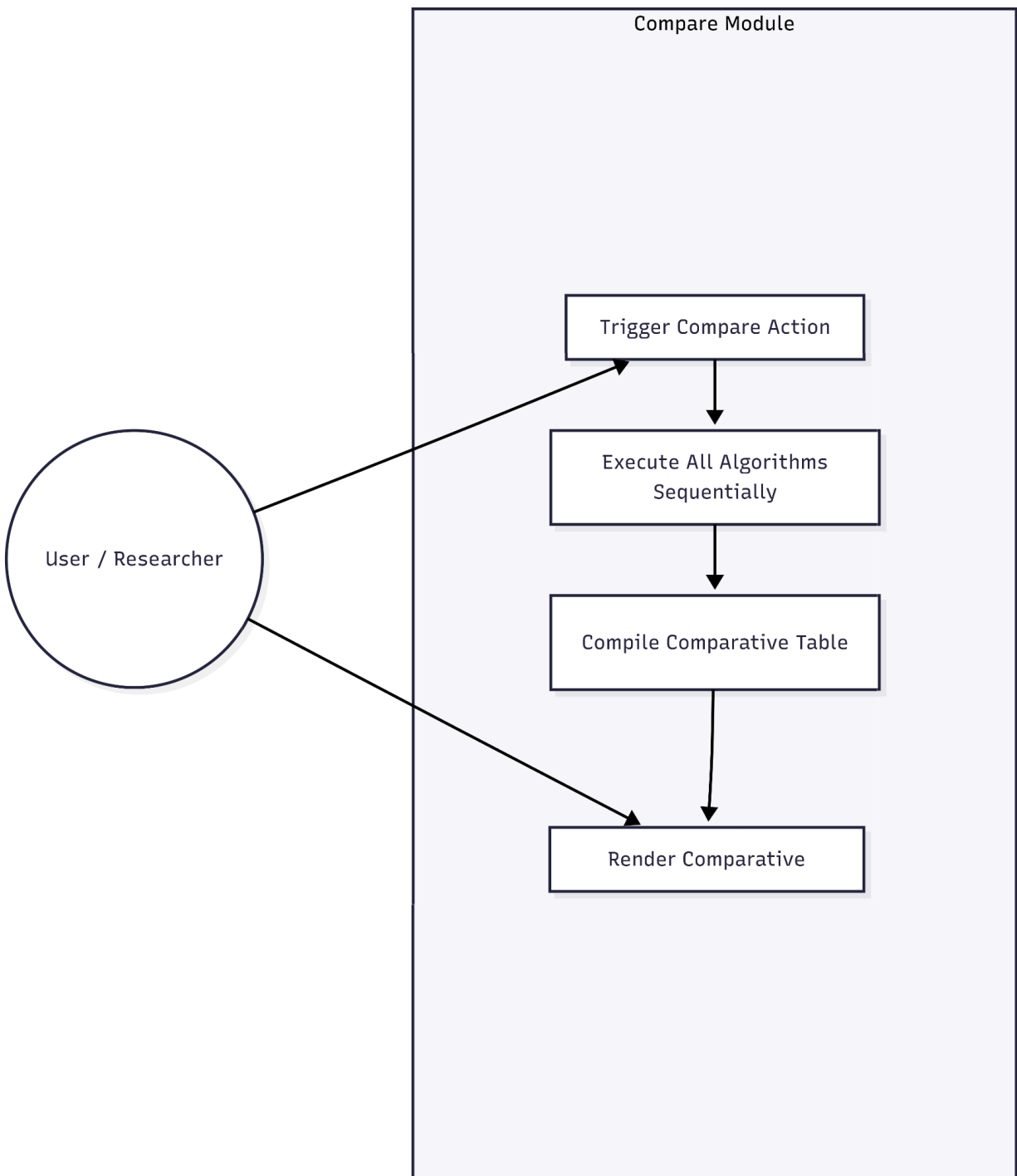


Figure 3.1c-5: Level 2 – Compare Module

Primary Actor: User / Researcher

Goal in Context: Breakdown of the Compare module — evaluating multiple decomposition methods side by side.

1. The user triggers the Compare action → system to run every supported algorithm.
2. The system sequentially executes all supported algorithms (CP, Tucker, HOSVD, TT, SVD, QR, LU).
3. Backend compiles a side-by-side table of runtimes, errors, and compression ratios.
4. Frontend renders comparative benchmark charts for the user.

3.3 Activity Diagram & Description

The Activity Diagram models the dynamic operational workflow of the system from user tensor submission through validation, algorithmic execution, error analysis, benchmarking, and interactive SVG visualization rendering.

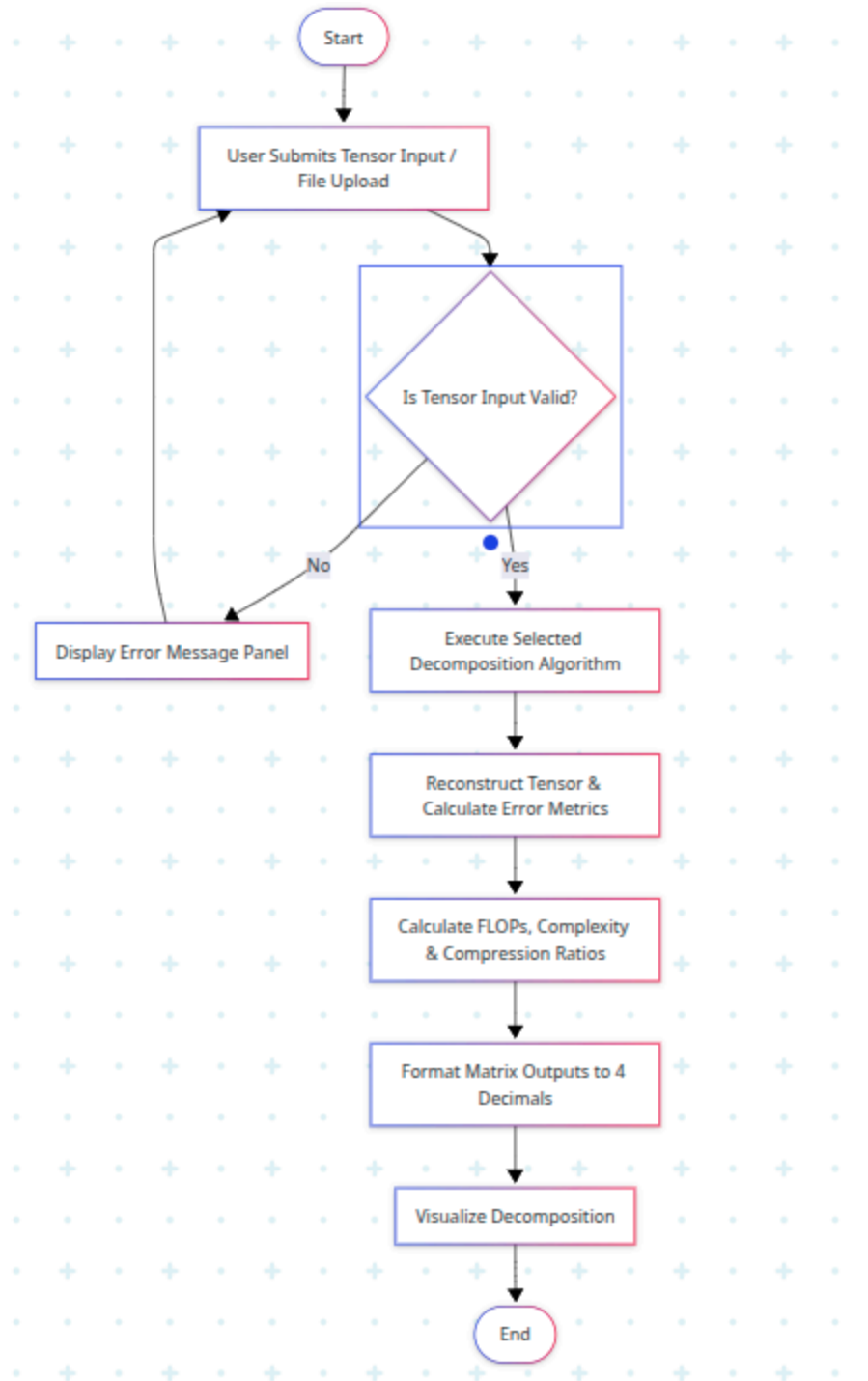


Figure 3.2: Tensor Decomposer System Activity Workflow

7. Preliminary Test Plan

7.1 Testing Objectives

The primary objectives of the preliminary testing phase are:

1. Algorithmic Precision Verification: Ensure matrix and tensor factorizations (CP-ALS, HOOI, HOSVD, TT, SVD, QR, LU) yield numerical solutions matching mathematical ground truth within 1e-6 error margins.
2. Input Validation Robustness: Verify that invalid array formats, malformed JSON files, mismatched matrix dimensions, and empty inputs trigger clean user-facing error panels without application crash.
3. Performance Analytics Validation: Confirm that single-run FLOPs estimations, compression ratio calculations, and execution duration measurements are consistent and accurate.
4. Frontend Interactivity & UI Stability: Ensure theme toggling, mutual exclusion input handling, loading skeleton overlays, and dynamic SVG visual cards render reliably across target browsers.

7.2 Features to be Tested

The preliminary high-level test suite specifies test scenarios, inputs, expected outputs, and acceptance criteria across all system modules.

Test ID	Feature Tested	Test Scenario & Input	Expected System Output	Status
TC-01	Manual Tensor Validation	Enter valid 3D array [[[1,2],[3,4]],[[5,6],[7,8]]].	The system accepts input, parses 2x2x2 shape.	PASS
TC-02	Invalid Text Handling	Enter a malformed string '[[1, 2], [invalid]]'.	System catches syntax error, displays clear error panel.	PASS
TC-03	Mutual Exclusion Input	Select file upload while the text field contains tensor data.	The text field automatically clears upon file upload.	PASS
TC-04	CP Decomposition (CP-ALS)	Execute CP on a 3x3x3 tensor with target rank R=2.	Returns weights vector and 3 factor matrices (3x2).	PASS

TC-05	Tucker Decomposition (HOOI)	Execute Tucker on a 3x4x5 tensor with ranks [2,2,2].	Returns 2x2x2 core tensor and 3 factor matrices.	PASS
TC-06	HOSVD Decomposition	Execute HOSVD on 4x4x4 tensor.	Returns orthogonal factor matrices and core tensor.	PASS
TC-07	Tensor Train Decomposition	Execute Tensor Train on 3D tensor.	Returns sequence of 3D TT core tensors.	PASS
TC-08	Matrix SVD Execution	Execute SVD on 4x4 matrix input.	Returns U (4x4), S spectrum (4), V ^H (4x4).	PASS
TC-09	Matrix QR Execution	Execute QR on 4x3 matrix.	Returns orthogonal Q (4x4) and upper-triangular R (4x3).	PASS
TC-10	Matrix LU Execution	Execute LU decomposition on square matrix.	Returns lower L and upper U matrices.	PASS
TC-11	Accuracy Metrics Calculation	Run Analysis on CP decomposed tensor.	Calculates MAE, RMSE, Relative Error, and reconstructed head.	PASS
TC-12	FLOPs & Benchmarking	Run Benchmark on Tucker decomposition.	Outputs execution time (ms), FLOPs string, compression ratio.	PASS
TC-13	Algorithmic Comparison	Trigger 'Compare' action on 3D tensor input.	Renders a comparative summary table for all algorithms.	PASS

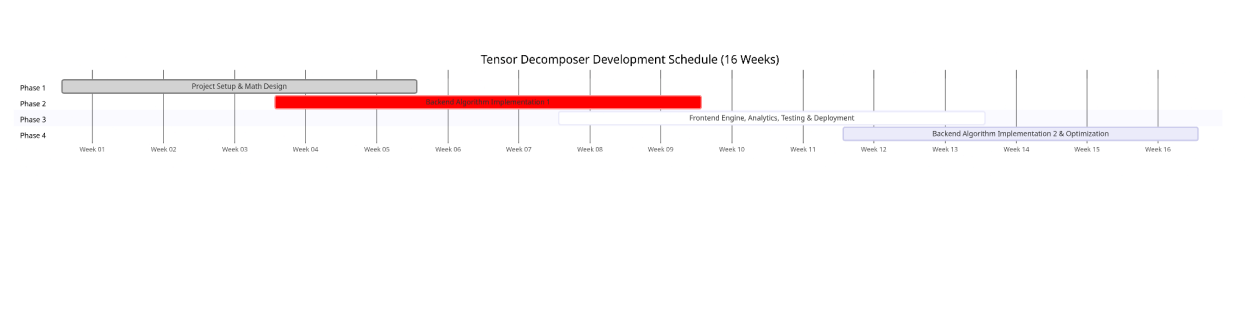
TC-14	3D Isometric Visualization	Trigger 'Visualize' action on 3D tensor.	SVG engine renders 3D scaled block equation diagram.	PASS
TC-15	Dark/Light Theme Toggle	Click the theme toggle button in header navigation.	Applies dark/light theme CSS tokens instantly & persists.	PASS

8. Timeline

8.1 Project Gantt Chart

The project development timeline spans a 16-week schedule across requirements analysis, algorithm implementation, frontend visualization engine development, benchmarking integration, testing, and final documentation.

Figure 8.1: Tensor Decomposer Project Development Gantt Chart (16 Weeks)



8.2 Work Breakdown Schedule

Project Phase	Timeline Schedule	Key Deliverables & Completed Milestones
Phase 1: Project Setup & Math Design	Weeks 1 – 5	Finalized problem requirements, defined multilinear tensor algorithms (CP-ALS, HOOI), and completed architecture design.
Phase 2: Backend Algorithm Implementation 1	Weeks 4 – 9	Implemented tensor decomposition algorithms (cp.py, tucker.py) and matrix factorizations.
Phase 3: Frontend Engine & ES Module Refactor, Analytics, Testing & Deployment	Weeks 8 – 13	Architected responsive UI, built dynamic SVG visualizer (3D isometric diagrams, heatmaps), and refactored home.js into ES modules.

		Integrated FLOPs benchmarking engine, precision error metrics (MAE/RMSE), executed test suite, and authored midterm technical report.
Phase 4: Backend Algorithm Implementation 2, Optimization and Library	Weeks 12 – 16	Will implement tensor decomposition algorithms (hosvd.py, tensort_train.py) and optimization. Will make a python library for local experimenting.

9. Conclusion

The Tensor Decomposer platform successfully addresses the critical need for a unified, accessible, and high-performance system to analyze, compress, benchmark, and visualize higher-dimensional tensor datasets. By implementing standard multilinear algorithms—including CP-ALS, HOOI/Tucker-ALS, HOSVD, and Tensor Train Decomposition—alongside traditional matrix factorizations, the platform provides research-grade decomposition capabilities accessible via a modern web interface.

The integration of single-run computational benchmarking (FLOPs estimation, complexity formulas, parameter compression ratios) and precision reconstruction error analysis (MAE, RMSE, Relative Error) empowers researchers to make informed trade-offs between compression efficiency and numerical accuracy. Furthermore, the custom SVG rendering engine delivers intuitive 3D isometric structural equations and component spectra.

Future Roadmap & Planned Enhancements:

1. Optimization: Further optimization on each algorithm based on puzzle tensor.
2. Tensor Decompose Library: Building a python library for each of tensor decompose algorithms and analysis functionalities for everyone to install and experiment locally.

10. References

1. Kolda, T. G., & Bader, B. W. (2009). Tensor decompositions and applications. *SIAM Review*, 51(3), 455-500.
3. Oseledets, I. V. (2011). Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5), 2295-2317.
4. Harshman, R. A. (1970). Foundations of the PARAFAC procedure: Models and conditions for an 'explanatory' multimodal factor analysis. *UCLA Working Papers in Phonetics*, 16, 1-84.
5. Tucker, L. R. (1966). Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3), 279-311.